

Системное ПО, роль системного ПО, низкоуровневое программирование

Зачем нужно системное ПО

- Его нет

Что такое системное ПО

Системное ПО — это комплекс программ, которые управляют аппаратными ресурсами компьютера и обеспечивают среду для запуска и выполнения прикладного софта

Какое бывает

- ОС
- Драйвер устройств
- Утилиты и сервисы
- Инструментальное ПО

Архитектура

4. Прикладное ПО

3. Среда выполнения и API

2. Системное ПО

1. Аппаратное обеспечение

Нулевой уровень абстракции

Ассемблер — это текстовая человекочитаемая форма машинного кода. Здесь нет понятий «класс», «цикл for» или «функция».

Основные сущности — это регистры процессора (RAX, RBX, RSP), оперативная память по конкретным адресам и стек.

Каждая строка кода на NASM транслируется почти ровно в одну машинную команду

Нулевой уровень абстракции

- **Плюсы:** Абсолютный контроль над процессором. Можно выжать 100% скорости из железа и написать программу размером в несколько килобайт.
- **Минусы:** Код, написанный для процессора Intel (x86/x64), не запустится на процессоре смартфона (ARM). Ошибиться очень легко: забыли очистить стек — программа сломалась.

Системный язык высокого уровня

C++ — уникальный гибрид и совмещает в себе абстракции высокого уровня, с другой стороны он компилируется прямо в машины код для конкретной ОС и процессора

- Код работает с типизированными переменными, объектами и указателями.
- В C++ нет автоматического сборщика мусора.
- На C++ пишутся ядра операционных систем (Windows, Linux, macOS), драйверы, игровые движки и СУБД.

Платформенный/Прикладной уровень

C# создан для того, чтобы программист думал о бизнес-логике приложения, а не о байтах и регистрах. Он работает на базе платформы .NET и виртуальной машины CLR

- Код на C# компилируется не в машинный код, а в промежуточный байт-код
- Garbage Collector

Платформенный/Прикладной уровень

Плюсы: Кроссплатформенность, безопасность и высокая скорость разработки.

Минусы: Потребляет больше ОЗУ. Сборщик мусора может внезапно заморозить программу на доли секунды для уборки, что недопустимо в реальном времени

Управление памятью и указатели

ASM/NASM

В ассемблере нет переменных в привычном смысле. Есть регистры процессора и адреса в оперативной памяти.

- **Стек:** Управляется вручную через регистр RSP. Команды push и pop физически двигают этот указатель вверх и вниз. Разработчик обязан следить, чтобы баланс стека не нарушился к моменту выхода из функции ret.
- **Доступ к памяти:** Происходит через разыменовывание регистров. Например, mov rax, [rbx]. Это чистейшая работа с адресами.

Управление памятью и указатели

C++

C++ дает полный контроль над обоими типами памяти и вводит понятие указателя

```
// Выделение памяти на стеке
int stackArray[100];

// Выделение памяти на куче
int* heapArray = new int[100];

// Арифметика указателей
int* secondElement = heapArray + 1; // Сдвиг на sizeof(int)
байт вперед

delete[] heapArray; // Если забыть – будет утечка памяти
```

Управление памятью и указатели

C#

- Ключевое слово `unsafe`: Разрешает использовать сырые указатели, как в C++.
- Конструкция `fixed`: Запрещает сборщику мусора перемещать объект в памяти, пока мы работаем с его адресом.

```
unsafe
{
    int[] numbers = { 10, 20, 30 };

    // Фиксируем массив в памяти, чтобы GC его не переместил
    fixed (int* p = numbers)
    {
        int* second = p + 1; // Арифметика указателей в C#
        Console.WriteLine(*second); // Выведет 20
    }
}
```


Взаимодействие между языками

Системный код редко пишется на одном языке. Обычно ядро или критичный по скорости модуль пишут на ASM/C++, а интерфейс или логику — на C#.

Взаимодействие между языками

NASM + C++

```
global asm_add ; Делаем функцию видимой снаружи  
  
section .text  
asm_add:  
    ; В x64 Windows: первый аргумент в RCX, второй в RDX  
    mov rax, rcx  
    add rax, rdx  
    ret ; Результат возвращается в регистре RAX
```

```
extern "C" long long asm_add(long long a, long long b); //  
Объявляем внешнюю функцию  
  
int main() {  
    long long result = asm_add(5, 7); // Вызов ассемблерного  
кода из C++  
    return 0;}
```

Взаимодействие между языками

C++ и C#

Платформа .NET умеет вызывать функции из скомпилированных динамических библиотек (.dll в Windows или .so в Linux)

```
using System;
using System.Runtime.InteropServices; // Необходим для работы
с нативным кодом

class Program
{
    // Импортируем функцию из системной DLL
    [DllImport("user32.dll", CharSet = CharSet.Unicode)]
    public static extern int MessageBox(IntPtr hWnd, string
text, string caption, uint type);

    static void Main()
    {
        // Вызываем низкоуровневое Win32 API окно из
управляемого C# кода
        MessageBox(IntPtr.Zero, "Привет из низкого уровня!",
"Системное ПО", 0);
    }
}
```


Суть

Низкоуровневое программирование — это не обязательно написание всей программы на ассемблере. Современный подход — писать 90% логики на удобном C#, а критические по скорости или работе с железом участки выносить в C++ или NASM и связывать их через P/Invoke и нативные вызовы.

Компилятор и компоновщик

Этап 1: Компиляция

Компилятор C++ (например, MSVC, GCC) или ассемблер (NASM) берет каждый файл исходного кода (.cpp или .asm) по отдельности и переводит его в машинные инструкции. На выходе получаются объектные файлы (.obj в Windows, .o в Linux). Объектный файл — это уже машинный код, но его еще нельзя запустить.

Компилятор и компоновщик

Этап 2: Линковка

Компоновщик берет все объектные файлы, а также внешние библиотеки, и сшивает их вместе.

Главная задача компоновщика — разрешить внешние ссылки. Если в файле `main.crr` вызывается функция `asm_add` из файла `file.asm`, линкер находит точное местоположение кода `asm_add` и заменяет заглушку на реальный адрес перехода в памяти.

В результате получается готовый исполняемый файл (`.exe` или ELF-бинарник без расширения в Linux).

Платформенная компиляция

Критерий	C++ / NASM	C# (.NET)
Что на выходе сборки	Нативный машинный код под конкретную ОС/архитектуру	Промежуточный байт-код (IL)
Переносимость	Нет. Скомпилированный под Windows x64 код не запустится на Linux или ARM	Да. Один и тот же IL-код запустится везде, где установлена среда .NET
Когда происходит оптимизация	Во время компиляции проекта	Частично при сборке, частично прямо во время работы

Сравнение

Вывод приветствия на C# и NASM

```
System.Console.WriteLine("Hello, World!");
```

<https://github.com/AKT-Polina/System-Programming/blob/main/Lecture/helloworld.asm>

Сравнение Debug и Release

```
int square_sum(int n) {  
    int sum = 0;  
    for (int i = 0; i < n; ++i) {  
        sum += i * i;  
    }  
    return sum;  
}
```

Низкоуровневое программирование

Низкоуровневое программирование – это процесс создания программ с использованием языков программирования, которые позволяют разработчикам иметь прямой доступ к аппаратному обеспечению компьютера, такому как процессор, память и другие устройства.

Низкоуровневое программирование

Преимущества

Высокая скорость выполнения кода

Максимальный контроль аппаратного обеспечения

Повышенная портируемость

Работа с ресурсами

Возможность оптимизации

Разработка низкоуровневого программного обеспечения более надежная и безопасная

Низкоуровневое программирование

Недостатки

Более трудоемкая разработка

Недостаточная переносимость

Малая экономическая ценность

Узкая специализация

Низкоуровневое программирование

Низкоуровневое программирование продолжает оставаться актуальным в различных областях. Оно активно используется в разработке операционных систем и встроенных систем, таких как микроконтроллеры, применяемые в бытовой электронике и автомобилях. Благодаря возможности точного управления оборудованием, низкоуровневое программирование обеспечивает высокую производительность и малое потребление ресурсов.

Низкоуровневое программирование

Например, в сфере интернет вещей низкоуровневое программирование позволяет создавать компактные и энергоэффективные решения. В системах реального времени, где важна мгновенная реакция на события, использование низкоуровневого кода является обязательным. Кроме того, в области компьютерной безопасности написание кода на низком уровне помогает выявлять и устранять уязвимости, обеспечивая надежную защиту данных.

Области применения

- Микроконтроллеры и микропроцессоры
- Системы реального времени
- Операционные системы
- Графические и видео драйверы
- Игровые движки с высокой производительностью